

تقرير الأمان ومعالجة الثغرات الأمنية

الحمد لله والصلاة والسلام على رسول الله وعلى آله وصحبه ومن والاه.

تم بفضل الله مراجعة النظام بشكل دقيق وتطبيق أفضل ممارسات هندسة البرمجيات الآمنة (S-SDLC) لمعالجة كافة الثغرات البرمجية والتهئية الأمنية التي كانت متواجدة في النسخة القديمة من النظام.

يوضح هذا التقرير جميع الثغرات التي تم اكتشافها ومعالجتها، وطريقة المعالجة، والملفات والأكواد البرمجية المستخدمة في تحقيق ذلك.

1. جدول ملخص الثغرات والمعالجات الأمنية

#	الثغرة الأمنية	مستوى الخطورة	طريقة المعالجة	الملفات المتأثرة
1	تخزين كلمات المرور والأسرار بنصوص واضحة في الكود وقاعدة البيانات	حرج جداً	استخدام خوارزمية bcrypt لتشفير كلمات المرور، ونقل جميع كلمات المرور والمفاتيح السرية إلى ملف تكوين خارجي .env .	config.py initial_data.sql auth_service.py
2	تخزين التوكنات بشكل غير آمن في المتصفح (XSS Risk)	مرتفع	حفظ الـ refresh_token في ملف تعريف ارتباط httpOnly آمن، والاحتفاظ بالـ access_token في ذاكرة التطبيق المؤقتة فقط.	tokenStore.js authService.js axiosClient.js
3	ثغرة إعادة استخدام Token (Replay Attack)	متوسط	تطبيق تدوير الرمز المميز (Refresh Token Rotation)، حيث يتم إبطال الرمز المستخدم فوراً وإصدار رمز جديد.	auth.py init.py
4	غياب فحص الصلاحيات وثغرات التحكم بالوصول (& IDOR Broken RBAC)	حرج جداً	إدراج فحص الصلاحيات من خلال ديكوريتور role_required على مستوى جميع المسارات الحساسة، والتأكد من عدم وصول الموظف لبيانات الآخرين.	rbac.py employees.py payroll.py
5	رفع الملفات غير المقيد (Unrestricted File Upload)	حرج جداً	فرض التحقق الثنائي (الامتداد + البنية الداخلية للملف عبر Pillow) وتوليد أسماء ملفات عشوائية فريدة لمنع رفع ملفات خبيثة كصور.	file_upload.py init.py
6	غياب رؤوس الأمان (Missing HTTP Security Headers & CSP)	متوسط	تفعيل حزمة Flask-Talisman لتحديد سياسة صارمة لمحتوى الصفحة (CSP) لبيانات الـ API وحماية التطبيق من Clickjacking.	init.py
7	هجمات التخمين وتكرار الطلبات (Brute-Force & DoS)	متوسط	تطبيق محدد الطلبات Flask-Limiter لتقييد محاولات تسجيل الدخول بـ 5 محاولات في الدقيقة كحد أقصى.	auth.py
8	كشف معلومات النظام عبر رسائل الخطأ التفصيلية (Information Disclosure)	منخفض	استبدال معالجة الأخطاء الافتراضية بمعالج مخصص يعيد رسائل عامة آمنة للتعامل مع الاحتفاظ بالتفاصيل في سجلات السيرفر المغلقة.	errors.py
9	غياب سجل التدقيق والمحاسبة (No Audit Logging)	مرتفع	إنشاء نظام تتبع متكامل audit_logs يسجل هوية المستخدم والعملية والوقت وعنوان الـ IP لأي عملية تعديل أو إضافة أو حذف.	audit.py

2. التفاصيل التقنية للثغرات وكود المعالجة

الثغرة الأولى: تخزين الأسرار وكلمات المرور بنصوص واضحة (Plaintext Credentials)

- **المشكلة:** كانت النسخة السابقة تحتفظ ببيانات الاتصال بقاعدة البيانات وكلمة مرور Gmail للمرسل البريدي مكتوبة بشكل صريح في الكود. كذلك كانت كلمات مرور السيرفر وقاعدة البيانات تُحفظ بدون تشفير، مما يسهل سرقتها بمجرد الوصول للملفات أو عبر مستودع الأكواد (Git).
- **طريقة المعالجة:**
- قمنا بنقل كافة الإعدادات الحساسة إلى ملف `env`. مستبعد تماماً من الرفع على Git.
- تم تطبيق تشفير كلمات المرور باستخدام خوارزمية **bcrypt** مع معامل تعقيد مناسب (`gensalt()`).
- **الكود البرمجي المستخدم:**

في ملف التكوين `config.py`:

```
python SECRET_KEY = os.getenv("SECRET_KEY") SQLALCHEMY_DATABASE_URI = os.getenv("DATABASE_URL")
JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY") MAIL_PASSWORD = os.getenv("MAIL_PASSWORD")
```

في خدمة التحقق `auth_service.py` (أثناء إنشاء الحساب وتشفيره):

```
python password_hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
```

في بذور قاعدة البيانات `initial_data.sql` (تم استبدال الكلمات الصريحة بتشفير **bcrypt**):

```
-- sql -- حساب مدير النظام (Admin) - كلمة المرور المشفرة لـ Admin@123456789
INSERT INTO users (username, email, password, role, status) VALUES ('admin', 'admin@example.com',
'$2b$12$MWGzntg96G23nQFJPQ.yQeVHAleC7A5jfMqGMMkaOSs68Msj15pgS', 'admin', 'active');
```

الثغرة الثانية: التخزين غير الآمن للرموز المميزة (Insecure Token Storage)

- **المشكلة:** كان الـ `access_token` والـ `refresh_token` يُخزنان بالكامل في الـ `sessionStorage` بالمتصفح. في حال تعرض الموقع لهجوم حقن البرمجة عبر المواقع (XSS)، يمكن للمهاجم الوصول لهذه التوكنات وسرقة جلسة المستخدم بشكل كامل ومستمر.
- **طريقة المعالجة:**
- تم فصل التوكنات: الـ `refresh_token` يتم تسليمه من الخلفية عبر ملف تعريف ارتباط (Cookie) مغلق بخاصية `httpOnly` و `Secure` و `SameSite=Lax`. هذا يمنع كود JavaScript تماماً من قراءته.
- الـ `access_token` يتم حفظه داخل ذاكرة التطبيق (`tokenStore.js`) في React فقط، ويختفي بمجرد إغلاق أو تحديث الصفحة.
- يتم استعادة الجلسة بطريقة صامتة ومؤمنة في الخلفية فور تحميل التطبيق عبر إرسال طلب للمسار `auth/refresh/`.
- **الكود البرمجي المستخدم:**

في متجر التوكنات بالواجهة `tokenStore.js: javascript let accessToken = null; export function`

```
getAccessToken() { return accessToken; } export function setAccessToken(token) { accessToken = token; } export function clearAccessToken() { accessToken = null; }
```

في إعدادات السيرفر `config.py: python JWT_TOKEN_LOCATION = ["headers", "cookies"]`

```
JWT_REFRESH_COOKIE_NAME = "refresh_token_cookie" JWT_COOKIE_SECURE = True
# إرسال الكوكي عبر HTTPS فقط JWT_COOKIE_CSRF_PROTECT = True حماية ضد هجمات CSRF
```

الثغرة الثالثة: ثغرة إعادة استخدام الرموز (Refresh Token Rotation - RTR)

- **المشكلة:** إذا تمكن مهاجم بطريقة ما من تسريب الـ `refresh_token` فإنه يبقى قادراً على توليد صلاحيات دخول جديدة دون معرفة صاحب الحساب الحقيقي.
- **طريقة المعالجة:** تم تطبيق آلية تدوير الرموز المميزة (**Rotation**). عند كل عملية تجديد صلاحية (`auth/refresh/`)، يتم وضع الـ `refresh_token` المستخدم فوراً في قائمة الحظر (Blocklist)، وإصدار توكن جديد بديل عنه. إذا حاول المهاجم إعادة استخدام نفس الرمز المحظور، يقوم النظام فوراً برفض العملية وتسجيل تنبيه أمني في السجلات لوجود محاولة اختراق محتملة للجلسة.
- **الكود البرمجي المستخدم:**

```
في المسار auth.py: ``python
old_jti = get_jwt()["jti"] (تدوير) refresh الرمز الحالي فوراً
db.session.add(TokenBlocklist(jti=old_jti))
```

```
# إصدار الرموز الجديدة
new_access_token, new_refresh_token = AuthService.issue_tokens(user)
db.session.commit()
```

في معالج الرموز المحظورة بملف `init.py: python @jwt.revoked_token_loader def`

```
def revoked_token_callback(jwt_header, jwt_payload):
    if jwt_payload.get("type") == "refresh":
        db.session.add(AuditLog(
            # تسجيل محاولة استخدام رمز ملغي في الـ audit logs لإدارة والتحذير الأمني
            action="auth.revoked_refresh_token_reused", metadata_json="Possible stolen/replayed refresh token detected via rotation"
        ))
```

الثغرة الرابعة: غياب التحقق من الصلاحيات وثغرات التحكم بالوصول (IDOR & Broken RBAC)

- **المشكلة:** كان مسار تحديث الموظفين القديم `update.php` مفتوحاً للعامّة بدون أي فحص للصلاحيات. كما كان بإمكان أي موظف استعلام بيانات زملائه الشخصية أو كشف رواتبهم بمجرد تعديل المعرف الرقمي (ID) في مسارات الـ API.
- **طريقة المعالجة:**

- قمنا ببناء فحص صلاحيات متكامل يعتمد على الأدوار (Role-Based Access Control) باستخدام ديكوريتور `@role_required` للتحقق من هوية ودور مرسل الطلب المخزن داخل الـ JWT الموقع من السيرفر.
- تم تطبيق تحقق ذكي على مستوى استعلام الموظفين ورواتبهم؛ بحيث يمكن للمسؤول (admin/hr) رؤية الكل، بينما يُجبر المستخدم العادي على رؤية بياناته الخاصة فقط المطابقة لمعرفه الرقمي.
- الكود البرمجي المستخدم:

وسيط الصلاحيات rbac.py:

```
python def role_required(*allowed_roles): def decorator(fn): @wraps(fn)
def wrapper(*args, **kwargs): verify_jwt_in_request() claims = get_jwt() if claims.get("role")
not in allowed_roles: return jsonify(success=False, message="Unauthorized"), 403 return fn(*args,
**kwargs) return wrapper return decorator
```

التحقق من الصلاحيات وحماية المعرفات في employees.py:

```
python ``employees_bp.route("/", methods=["PUT"]) @role_required("admin",@
... : "hr") def update_employee(employee_id)
```

منع ثغرة IDOR عند طلب الملف الشخصي لموظف آخر employees_bp.route("/", methods=["GET"])@login_required def get_employee(employee_id): claims = get_jwt
if claims.get("role") not in ("admin", "hr") and ... : "hr") def update_employee(employee_id)
claims.get("employee_id") != employee_id: return jsonify(success=False, message="Unauthorized"),
403

الثغرة الخامسة: رفع الملفات غير المقيد (Unrestricted File Upload)

- **المشكلة:** كان النظام القديم يثق باسم وامتداد الملف المرسل من العميل كصورة للموظف، مما يفتح الباب لرفع نصوص برمجية خبيثة (Web Shells) كملفات بايثون أو PHP وتسميتها بملحقات وهمية مثل `png`. لتنفيذ أكواد عشوائية على الخادم (RCE).
- **طريقة المعالجة:**
- التحقق من امتداد الملف عبر قائمة بيضاء معتمدة (whitelist).
- فحص البنية البايثية الداخلية للملف للتأكد من كونه صورة صالحة فعلياً عبر مكتبة **Pillow** لتفادي الخدع الأمنية.
- تغيير اسم الملف المرفوع لاسم فريد عشوائي (UUID) لمنع هجمات Path Traversal ولمنع السيطرة على مسار الحفظ أو معرفة الروابط وتخمينها.
- فرض التحقق من الـ JWT للوصول للملفات المرفوعة عبر مسار خاضع للصلاحيات مع إرفاق ترويسة الحماية `X-Content-Type-Options: nosniff`.
- الكود البرمجي المستخدم:

في وحدة رفع الملفات file_upload.py: ``python def _looks_like_real_image(file_storage): try: from PIL import Image file_storage.stream.seek(0) with Image.open(file_storage.stream) as img: img.verify
()return True except Exception: return False فحص بنية وبايتات الصورة الداخلية

filename = secure_filename(file_storage.filename) ext = filename.rsplit(".", 1)[-1].lower() if "." in filename else ""
:def save_uploaded_file(file_storage, subfolder, allowed_extensions) # التحقق من الامتداد
"if ext not in allowed_extensions: return None, f"File type {ext} is not allowed"

```
# فحص محتوى البايتات للصورة
if ext in IMAGE_EXTENSIONS and not _looks_like_real_image(file_storage):
    return None, "File content does not match a valid image"

# توليد اسم فريد آمن
unique_name = f"{uuid.uuid4().hex}.{ext}"
...
```

...

في ملف البدء init.py: python @app.route("/api/uploads/<path:filename>") def serve_upload(filename):

```
verify_jwt_in_request() # لا يمكن لأي زائر غير مسجل رؤية الملفات المرفوعة
response = send_from_directory(app.config["UPLOAD_FOLDER"], filename)
response.headers["X-Content-Type-Options"] = "nosniff" return response المتصفح لنوع الملف
```

الثغرة السادسة: تفعيل رؤوس الأمان (Security Headers & CSP)

- **المشكلة:** عدم إرسال ترويسات الأمان الافتراضية لحماية بيانات الواجهة الأمامية والخلفية من الهجمات التفاعلية ك Clickjacking أو حظر إطارات العرض الخارجي للموقع.
- **طريقة المعالجة:** تفعيل مكتبة Flask-Talisman مع تحديد سياسة صارمة لتحديد مصادر المحتوى (Content Security Policy) الخاصة ببيانات API بحيث لا تسمح بعرض صفحات أو تنفيذ نصوص برمجية من نطاقات أخرى، وتجبر التطبيق في وضع الإنتاج على العمل الحصري بنظام الاتصال الآمن (HTTPS).
- **الكود البرمجي المستخدم في init.py:**

```
python csp = {"default-src": "none", "frame-ancestors": "none"} talisman.init_app( app, force_https=app.config.get("FORCE_HTTPS", False),
content_security_policy=csp, )
```

الثغرة السابعة: محدد المحاولات والحماية من التخمين (Rate Limiting)

- **المشكلة:** كان بإمكان المهاجم تجربة كلمات مرور لانهائية على مسار تسجيل الدخول دون قيود، مما يجعل كشف الحسابات الضعيفة مسألة وقت.
- **طريقة المعالجة:** تثبيت مكوّن `Flask-Limiter` لربط مسار تسجيل الدخول بعدد محدد من الطلبات للشخص الواحد (IP Address).
- **الكود البرمجي المستخدم في** `auth.py`:

```
python @auth_bp.route("/login", methods=["POST"])
...
@limiter.limit("5 per minute")
def login():
    ...
```

الثغرة الثامنة: معالجة الأخطاء والتسريب البرمجي (Safe Error Handling)

- **المشكلة:** في حالة حدوث خلل برمجي أو استثناء غير متوقع، كان النظام يعيد رسالة الخطأ الخام للمتصفح بكل تفاصيلها (خطأ قاعدة البيانات، أسماء الأعمدة والجداول، مسارات الملفات)، وهي معلومات استخباراتية ثمينة للمخترقين.
- **طريقة المعالجة:** كتابة معالج أخطاء مخصص يستبدل الأخطاء التفصيلية بردود موحدة عامة، مع تحويل الأخطاء البرمجية الخطيرة داخلياً إلى ملف السجلات الفني الآمن.
- **الكود البرمجي المستخدم في** `errors.py`:

```
python def register_error_handlers(app):
    @app.errorhandler(404) def not_found(e): return jsonify(success=False, message="Not found"), 404

    @app.errorhandler(500) def server_error(e): app.logger.exception(e)
    return jsonify(success=False, message="Internal server error"), 500
```

الثغرة التاسعة: سجل التدقيق والمراقبة (Audit Logs)

- **المشكلة:** غياب سجلات تتبع توضح من قام بإنشاء موظف، أو من قام بتحديث راتب، أو من فشل في تسجيل الدخول، مما يؤدي إلى عدم القدرة على تحديد الفاعل والمسؤول عن العمليات الحساسة (Non-repudiation).
- **طريقة المعالجة:** قمنا بإنشاء جدول وسجل مخصص في قاعدة البيانات يتم استدعاؤه فور تنفيذ أي عملية حساسة (إنشاء، تعديل، حذف، تسجيل دخول ناجح/فاشل) ويسجل هوية الفاعل، دوره، طبيعة العملية، عنوان الـ IP الخاص به، والمعرف المستهدف.
- **الكود البرمجي المستخدم في** `audit.py`:

```
python def log_action(action, target_type=None, target_id=None, metadata=None):
    try:
        claims = get_jwt()
        actor_id = int(get_jwt_identity())
        if
```

```
get_jwt_identity() else None actor_role = claims.get("role") except RuntimeError: actor_id,  
actor_role = None, None
```

```
entry = AuditLog( actor_user_id=actor_id, actor_role=actor_role, action=action,  
target_type=target_type, target_id=target_id, ip_address=request.remote_addr if request else  
None, metadata_json=str(metadata) if metadata else None, ) db.session.add(entry)  
```` db.session.commit
```

---

**ختاماً:** بفضل هذه الإصلاحات، أصبح تطبيق الويب يتمتع ببنية تحتية أمنية ممتازة تحميه من معظم هجمات الويب الشائعة (OWASP Top 10) وتضمن سرية وسلامة البيانات الحساسة مثل معلومات الموظفين والرواتب والعمليات المالية. صلى الله وسلم على نبينا محمد وعلى آله وصحبه أجمعين.